

MRPL Sovereign AI Workbench — Environment & Tech Stack

1. Project Overview

MRPL Sovereign AI Workbench is a local, air-gapped AI workbench designed to operate without external AI APIs or cloud dependencies.

The system uses a local LLM/VLM runtime, an MCP-based tool layer, local RAG storage, document-generation tools, a Docker-isolated code sandbox, and a Streamlit frontend with an OpenCode-inspired developer-workbench design.

Core principles

- Local-first / air-gapped operation
- No external LLM API dependency
- MCP-based tool integration
- Lightweight local models suitable for a 6 GB VRAM target
- Sandboxed code execution
- Observable request/tool lifecycle
- OpenCode-inspired frontend experience

2. Complete Tech Stack

2.1 Programming Language

Technology Version / Target		Purpose
Python	3.11+	Main application, orchestration, MCP servers, utilities

2.2 Frontend

Technology	Purpose
Streamlit	Web-based application UI
Custom CSS / Streamlit components	OpenCode-inspired visual design

Technology
OpenCode-inspired UX

Purpose
Developer/workbench visual language

Frontend design direction

The UI should NOT look like a default Streamlit chatbot.

It should use:

- Left-side workspace/navigation
- Main conversation/work area
- Tool execution trace
- Model/status badges
- Offline/network status
- Generated-file panels
- Clear task/tool states
- Restrained colors
- Developer-tool/workbench visual hierarchy
- Compact technical UI elements

Important boundary

OpenCode is a **frontend design reference**, not a backend dependency.

The application frontend remains:

```
Streamlit
+
Custom CSS/components
+
OpenCode-inspired design
```

3. AI / Model Stack

3.1 Local Model Runtime

Ollama

Purpose:

- Local LLM/VLM serving

- Model management
- Local inference
- No external AI API required

Default endpoint:

http://localhost:11434

3.2 Models

Model	Role	Target
llama3.2:3b	General chat / reasoning	Lightweight local inference
qwen2.5-coder:3b	Code generation	Code-related tasks
moondream	Vision/document understanding	Image/vision fallback

Model configuration

Use `models_config.json` as the single source of truth for model names and inference configuration.

Recommended configuration:

```
{
  "chat": {
    "model": "llama3.2:3b",
    "num_ctx": 4096
  },
  "code": {
    "model": "qwen2.5-coder:3b",
    "num_ctx": 4096
  },
  "vision": {
    "model": "moondream",
    "num_ctx": 4096
  }
}
```

Model-selection rule

Application code should obtain model names through the centralized configuration instead of hard-coding model names in multiple files.

4. Agent / Orchestration Layer

Python Orchestrator

The orchestrator controls the complete request lifecycle.

Typical flow:

```
User Request
  ↓
Intent Gate
  ↓
Task Router
  ↓
Fixed One-Shot Flow
    OR
ReAct / Iterative Flow
  ↓
MCP Tool Layer
  ↓
Local Model Layer
  ↓
Result / Generated Artifact
  ↓
Streamlit UI
```

Main responsibilities

- Request classification
 - Intent gating
 - Task routing
 - Model selection
 - Tool selection
 - MCP communication
 - Iterative agent loop
 - Error handling
 - Tool tracing
 - Final response generation
-

5. MCP Stack

MCP / FastMCP

MCP provides the standard interface between the orchestrator and specialized tools.

Recommended implementation:

`FastMCP`
`+`
`Python MCP servers`

MCP servers

MCP Server	Responsibility
<code>rag_mcp_server.py</code>	Local document retrieval
<code>vision_mcp_server.py</code>	OCR and vision processing
<code>docgen_mcp_server.py</code>	DOCX generation
<code>xlsx_mcp_server.py</code>	Excel generation
<code>pptx_mcp_server.py</code>	PowerPoint generation
<code>email_mcp_server.py</code>	Local email delivery
<code>code_exec_mcp_server.py</code>	Sandboxed code execution

6. RAG Stack

Technology	Purpose
ChromaDB	Local vector database
Sentence Transformers	Text embeddings
Python	Ingestion/retrieval pipeline

RAG flow

```
Documents
  ↓
Text extraction
  ↓
Chunking
  ↓
Sentence Transformer embeddings
  ↓
ChromaDB
  ↓
Similarity search
  ↓
Relevant context
  ↓
Local LLM
```

RAG configuration

Default:

```
RAG_TOP_K=3
```

The RAG system must operate locally.

7. OCR / Vision Stack

Technology	Purpose
Tesseract OCR	Primary OCR
PyTesseract	Python interface to Tesseract
Pillow	Image processing
Moondream	Vision fallback

Vision flow

```
Image / Document
  ↓
Tesseract OCR
  ↓
Is extracted text sufficient?
  /      \
YES       NO
  ↓       ↓
Return text Moondream
          ↓
        Vision analysis
```

This provides a lightweight OCR-first strategy while retaining a local vision fallback.

8. Document Generation Stack

DOCX

```
python-docx
```

Purpose:

- Generate Word documents
- Reports
- Letters
- Structured office documents

XLSX

```
openpyxl
```

Purpose:

- Generate Excel workbooks
- Tables
- Structured datasets

- Spreadsheet reports

PPTX

```
python-pptx
```

Purpose:

- Generate PowerPoint presentations
- Slide decks
- Structured presentation output

9. Email Stack

Local email architecture

```
Application
  ↓
Python smtplib
  ↓
localhost:1025
  ↓
MailHog
  ↓
MailHog Web UI
```

SMTP

```
Host: localhost
Port: 1025
```

MailHog is used as a local email sink for development/demo purposes.

No external SMTP provider is required.

10. Code Execution Sandbox

Docker

Docker provides isolated execution for generated code.

Recommended restrictions:

```
Network: disabled
Memory: 256 MB
CPU: 0.5 CPU
Timeout: 15 seconds
```

Sandbox flow

```
Generated Code
  ↓
code_exec_mcp_server.py
  ↓
Docker container
  ↓
--network none
  ↓
Resource limits
  ↓
Execution
  ↓
Output / Error
```

Default image

```
python:3.11-slim
```

The sandbox should never be treated as unrestricted host execution.

11. System Monitoring

Recommended Python dependency:

```
psutil
```

Purpose:

- CPU monitoring
- Memory monitoring
- Process/system status
- Runtime diagnostics

The frontend can expose lightweight system/runtime status where useful.

12. Configuration

Primary configuration

```
models_config.json
```

Use this for:

- Model names
- Context sizes
- Model roles
- Model-specific inference settings

Environment configuration

```
.env
```

Use `.env` for:

- Local machine/runtime settings
- Ports
- Paths
- Resource limits
- Feature flags

Do not store model definitions redundantly in multiple places.

13. `.env.example`

```
# =====
# MRPL SOVEREIGN AI WORKBENCH
# Local Environment Configuration
# =====

# -----
# Application
# -----

APP_HOST=localhost
APP_PORT=8501

# -----
# Ollama
# -----

OLLAMA_HOST=http://localhost:11434

# -----
# Storage
# -----

DATA_DIR=./sample_data
GENERATED_FILES_DIR=./generated_files
CHROMA_DIR=./chroma_db

# -----
# RAG
# -----

RAG_TOP_K=3

# -----
# Agent / Orchestrator
# -----

MAX_REACT_ITERATIONS=5
NUM_CTX=4096

# -----
# MailHog
# -----
```

```
SMTP_HOST=localhost
SMTP_PORT=1025
MAILHOG_WEB_PORT=8025
```

```
# -----
# Docker Code Sandbox
# -----
```

```
DOCKER_IMAGE=python:3.11-slim
SANDBOX_TIMEOUT=15
SANDBOX_MEMORY=256m
SANDBOX_CPUS=0.5
```

```
# -----
# OCR
# -----
```

```
TESSERACT_CMD=tesseract
```

```
# -----
# Network Safety
# -----
```

```
OFFLINE_MODE=true
ALLOW_EXTERNAL_NETWORK=false
```

14. Python Dependencies

Recommended `requirements.txt`:

```
fastmcp
mcp
ollama
streamlit
chromadb
sentence-transformers
python-docx
python-pptx
openpyxl
pytesseract
Pillow
psutil
python-dotenv
numpy
```

Install with:

```
pip install -r requirements.txt
```

15. External Software Dependencies

The following are system-level dependencies rather than normal Python packages.

Ollama

Required for local model serving.

```
Ollama
└─ localhost:11434
```

Tesseract

Required for OCR.

Verify:

```
tesseract --version
```

Docker

Required for the code execution sandbox.

Verify:

```
docker --version
```

MailHog

Required for local email testing/demo.

Default:

```
SMTP: localhost:1025  
Web UI: localhost:8025
```

16. Models to Pull

Recommended lightweight model set:

```
ollama pull llama3.2:3b  
ollama pull qwen2.5-coder:3b  
ollama pull moondream
```

Verify:

```
ollama list
```

17. Local Ports

Service	Port	Purpose
Streamlit	8501	Main UI
Ollama	11434	Local LLM/VLM API

Service	Port	Purpose
MailHog SMTP	1025	Local SMTP
MailHog Web	8025	Email inspection

18. Security / Sovereignty Requirements

The project must follow these rules:

Required

- Local model inference
- Local vector database
- Local OCR
- Local document generation
- Local email sink
- Sandboxed code execution
- Network-disabled code containers
- No external AI API dependency

Avoid

```
OpenAI API
Groq API
Gemini API
Claude API
External SMTP
External vector database
Cloud code execution
```

No API keys should be required for the core demo.

19. Important Architecture Decision

Do not use the older heavyweight model configuration if the target machine has only 6 GB VRAM.

Avoid making these the default:

```
llama3.1:8b  
qwen2.5-coder:7b  
qwen2-vl:7b
```

Use the lightweight target stack:

```
llama3.2:3b  
qwen2.5-coder:3b  
moondream
```

This keeps the project aligned with the local-resource constraint.

20. Final Stack Summary

FRONTEND		
Streamlit + OpenCode-inspired UI		
ORCHESTRATOR		
Python + Intent Gate + Task Router		
+ ReAct Loop		
MCP		
FastMCP + Specialized Python MCP Servers		
RAG	Vision	Generation
ChromaDB	Tesseract	python-docx
Sentence	PyTesseract	openpyxl
Transformers	Pillow	python-pptx
	Moondream	
LOCAL AI RUNTIME		
Ollama		
llama3.2:3b qwen2.5-coder:3b moondream		
SAFE EXECUTION		
Docker + network disabled + resource limits		
EMAIL		
smtplib + MailHog		
MONITORING		
psutil		

21. One-Line Technology Stack

Python 3.11+ + Streamlit + OpenCode-inspired UI + Ollama + Llama 3.2 3B + Qwen 2.5 Coder 3B + Moondream + MCP/FastMCP + ChromaDB + Sentence Transformers + Tesseract + Pillow + python-docx + openpyxl + python-pptx + MailHog + Docker + psutil.

